

AASD 4010

Deep Learning - I

Applied AI Solutions Developer Program



LSTM



Agenda



- LSTM
- Training a LSTM
- Sequence Problem Types
- Model Types
- Which model to choose for a problem
- Vanilla LSTM
- Stacked LSTM
- CNN LSTM
- Encoder-Decoder LSTM
- Bidirectional LSTM



LSTM

Recurrent neural network is a type of neural networks which suffers from Short-term dependencies in its network

Solution: LSTM

LSTM (Long Short Term Memory network) is a variant of RNN that solves this issue



LSTM Input Layer

The LSTM input layer must be 3D.

The meaning of the 3 input dimensions are: ***samples, time steps and features***.

The LSTM input layer is defined by the input shape argument on the first hidden layer.

The input shape argument takes a tuple of two values that define the number of time steps and features.

The number of samples is assumed to be 1 or more [OPTIONAL]

The reshape() function on NumPy arrays is used to reshape your 1D or 2D data to be 3D.

The reshape() function takes a tuple as an argument that defines the new shape.



Training a LSTM



How to train a ML model ?

Backpropagation Algorithm

1. Present a training input pattern and propagate it through the network to get an output.
2. Compare the predicted outputs to the expected outputs and calculate the error.
3. Calculate the derivatives of the error with respect to the network weights.
4. Adjust the weights to minimize the error.
5. Repeat.



How to train a LSTM ?

Backpropagation Through Time (**BPTT**) is the application of the Backpropagation training algorithm to Recurrent Neural Networks. In the simplest case, a recurrent neural network is shown one input each time step and predicts one output.

Conceptually, BPTT works by unrolling all input time steps. Each time step has one input time step, one copy of the network, and one output. Errors are then calculated and accumulated for each time step. The network is rolled back up and the weights are updated.

1. Present a sequence of time steps of input and output pairs to the network.
2. Unroll the network then calculate and accumulate errors across each time step.
3. Roll-up the network and update weights.
4. Repeat.



Issues with BPTT

BPTT can be computationally expensive as the number of time steps increases. If input sequences are comprised of thousands of time steps, then this will be the number of derivatives required for a single weight update. This can cause weights to vanish or explode (go to zero or overflow) and make slow learning and model skill noisy.

Solution: **Truncated BPTT**

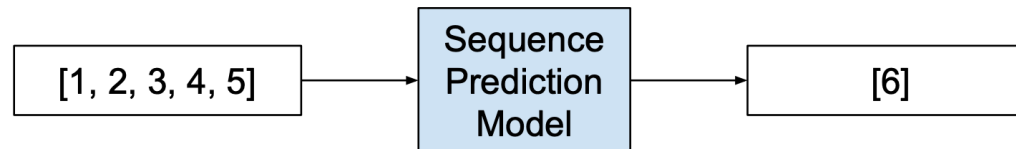
1. Present a sequence of k_1 time steps of input and output pairs to the network.
2. Unroll the network, then calculate and accumulate errors across k_2 time steps.



Sequence Problems



Sequence Prediction Problem



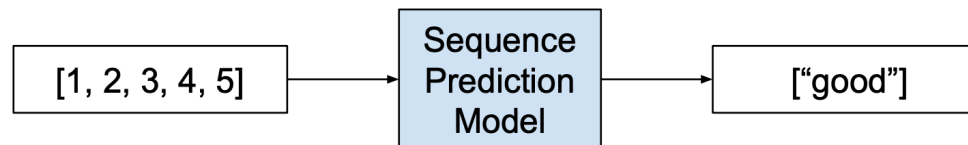
Weather Forecasting: Given a sequence of observations about the weather over time, predict the expected weather tomorrow.

Stock Market Prediction: Given a sequence of movements of a security over time, predict the next movement of the security.

Product Recommendation: Given a sequence of past purchases for a customer, predict the next purchase for a customer.



Sequence Classification Problem



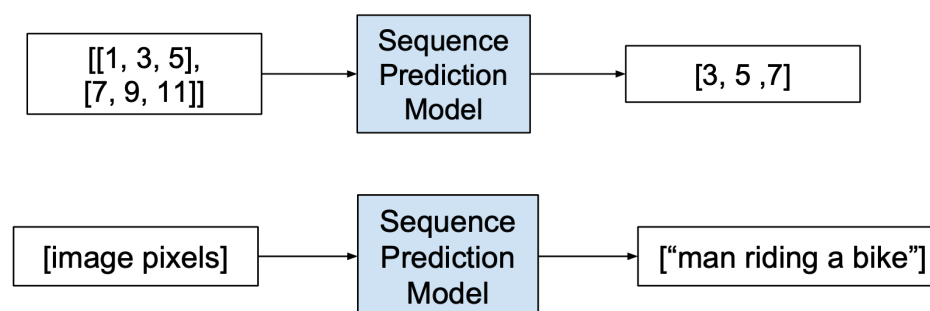
DNA Sequence Classification : Given a sequence of DNA, predict it as a coding or non-coding region.

Anomaly Detection : Given a sequence of observations, predict the sequence is anomalous or not.

Sentiment Analysis : Given a sequence of review text, tweet, financial news, predict the sentiment



Sequence Generation Problem



Text Generation: Given a corpus of text, such as the works of Shakespeare, generate new sentences or paragraphs of text that read they could have been drawn from the corpus.

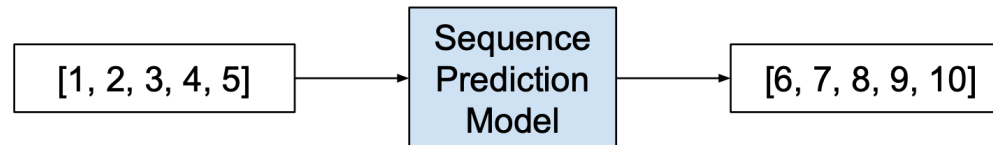
Handwriting Prediction: Given a corpus of handwriting examples, generate handwriting for new phrases that has the properties of handwriting in the corpus.

Music Generation: Given a corpus of examples of music, generate new musical pieces that have the properties of the corpus.

Image Caption Generation: Given an input image, generate a sequence of words that describe an image.



Sequence to Sequence Prediction Problem



Multi-Step Time Series Forecasting: Given a time series of observations, predict a sequence of observations for a range of future time steps.

Text Summarization: Given a document of text, predict a shorter sequence of text that describes the salient parts of the source document.

Program Execution: Given the textual description program or mathematical equation, predict the sequence of characters that describes the correct output.



Models



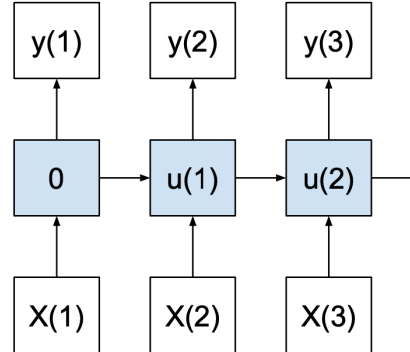
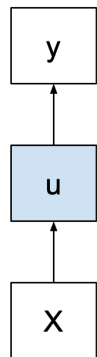
Model Types

One-to-One
Many-to-One
One-to-Many
Many-to-Many



One-to-One Model

$f(\dots)$ produces one output $y(t)$ value for each input value $X(t)$



Use: Sequence Generation

```

y(1) = f(X(1))
y(2) = f(X(2))
y(3) = f(X(3))
...

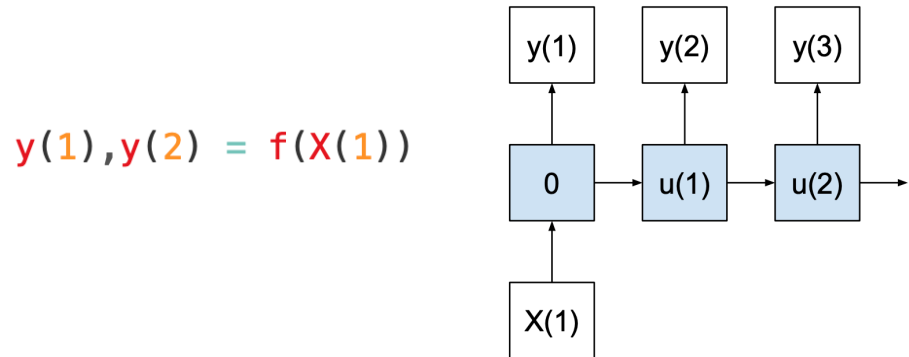
model = Sequential()
model.add(LSTM(..., input_shape=(1, ...)))
model.add(Dense(1))

```



One-to-Many Model

$f(\dots)$ produces multiple output values $y(t)$, $y(t+1)$ for one input value $X(t)$



Use: Image Caption Generation

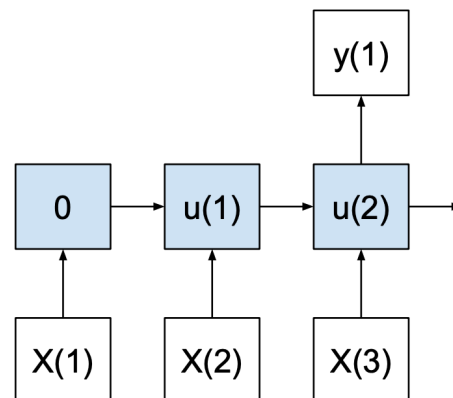
```
model = Sequential()
model.add(Conv2D(...))
...
model.add(LSTM(...))
model.add(TimeDistributed(Dense(1)))
```



Many-to-One Model

$f(\dots)$ produces one output $y(t)$ after receiving multiple input values $X(t), X(t+1)$

$$y(1) = f(X(1), X(2))$$



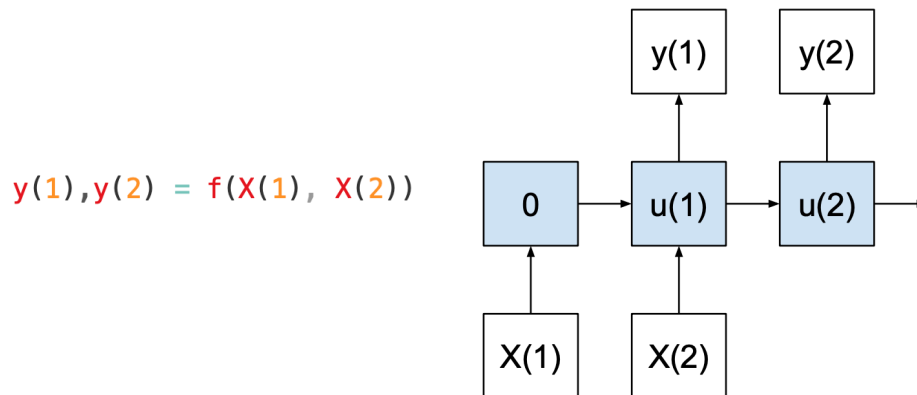
Use: Sentiment Analysis

```
model = Sequential()  
model.add(LSTM(..., input_shape=(steps, ...)))  
model.add(Dense(1))
```



Many-to-Many Model

$f(\dots)$ produces multiple outputs $y(t), y(t+1)$ after receiving multiple input values $X(t), X(t+1)$



Use: Summarization,
Translation

Encoder-Decoder

```
model = Sequential()
model.add(LSTM(..., input_shape=(steps, ...), return_sequences=True))
model.add(TimeDistributed(Dense(1)))
```

```
model = Sequential()
model.add(LSTM(..., input_shape=(in_steps, ...)))
model.add(RepeatVector(out_steps))
model.add(LSTM(..., return_sequences=True))
model.add(TimeDistributed(Dense(1)))
```



Which model to choose ?



Which model to choose ?

TIME SERIES APPLICATIONS

Univariate Time Series Forecasting: One series with multiple input time steps and wish to predict one time step beyond the input sequence.

Multivariate Time Series Forecasting: Multiple series with multiple input time steps and wish to predict one time step beyond one or more of the input sequences. Each series is just another input feature.

Multi-step Time Series Forecasting: One or multiple series with multiple input time steps and wish to predict multiple time steps beyond one or more of the input sequences.

Time Series Classification: One or multiple series with multiple input time steps as input and wish to output a classification label.



Which model to choose ?

NLP APPLICATIONS

Image Captioning: One image and wish to generate a textual description.

Video Description: Sequence of images in a video and wish to generate a textual description.

Sentiment Analysis: Sequences of text as input and you wish to generate a classification label.

Speech Recognition: Sequence of audio data as input and wish to generate a textual description of what was spoken.

Text Translation: Sequence of words in one language as input and wish to generate a sequence of words in another language.

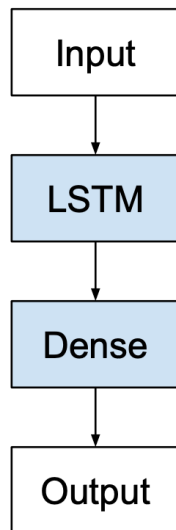
Text Summarization: Document of text as input and wish to create a short textual summary of the document as output.



Vanilla LSTM



Vanilla LSTM



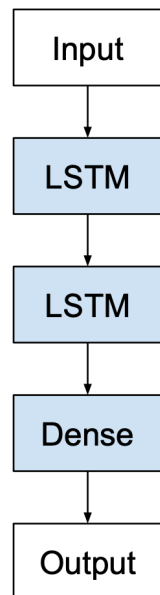
```
model = Sequential()  
model.add(LSTM(..., input_shape=(...)))  
model.add(Dense(...))
```



Stacked LSTM



Stacked LSTM



```
model = Sequential()  
model.add(LSTM(..., return_sequences=True, input_shape=(...)))  
model.add(LSTM(...))  
model.add(Dense(...))
```

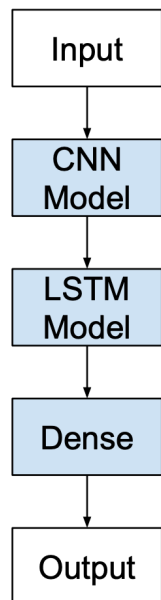
```
model = Sequential()  
model.add(LSTM(..., return_sequences=True, input_shape=(...)))  
model.add(LSTM(..., return_sequences=True))  
model.add(LSTM(..., return_sequences=True))  
model.add(LSTM(...))  
model.add(Dense(...))
```



CNN LSTM



CNN LSTM



```

cnn = Sequential()
cnn.add(Conv2D(...))
cnn.add(MaxPooling2D(...))
cnn.add(Flatten())
  
```

```

model = Sequential()
model.add(TimeDistributed(cnn, ...))
model.add(LSTM(...))
model.add(Dense(...))
  
```

```

model = Sequential()

# define CNN model
model.add(TimeDistributed(Conv2D(...)))
model.add(TimeDistributed(MaxPooling2D(...)))
model.add(TimeDistributed(Flatten(...)))

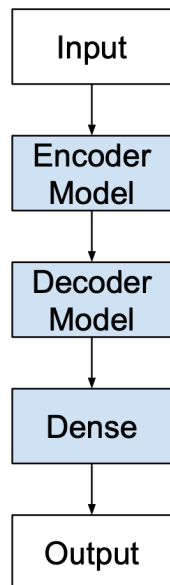
# define LSTM model
model.add(LSTM(...))
model.add(Dense(...))
  
```



Encoder-Decoder LSTM



Encoder-Decoder LSTM



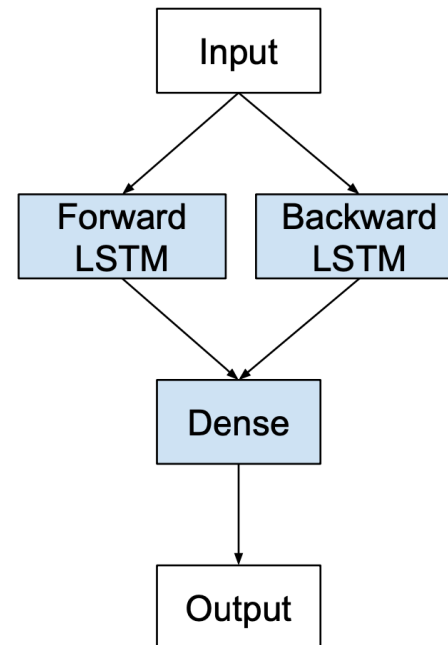
```
model = Sequential()  
# Encoder  
model.add(LSTM(..., input_shape=(...)))  
model.add(RepeatVector(...))  
  
# Decoder  
model.add(LSTM(..., return_sequences=True))  
model.add(TimeDistributed(Dense(...)))
```



Bidirectional LSTM



Bidirectional LSTM



```
model = Sequential()
model.add(Bidirectional(LSTM(50, return_sequences=True), input_shape=(n_timesteps, 1)))
model.add(TimeDistributed(Dense(1, activation='sigmoid')))
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['acc'])
model.summary()
```



Evaluating a LSTM



How to evaluate a LSTM ?

A LSTM model can be

- Underfit
- Overfit
- Good fit

Do Repeats of training

```
# Multiple repeats of the Stochastic LSTMs
for i in range(5): # Can vary
    # define model
    model = Sequential()
    model.add(LSTM(10, input_shape=(1,1)))
    model.add(Dense(1, activation='linear'))
    # compile model
    model.compile(loss='mse', optimizer='adam')
    X,y = get_train()
    valX, valY = get_val()
    # fit model
    history = model.fit(X, y, epochs=300,
                        validation_data=(valX, valY),
                        shuffle=False) # store history
    train[str(i)] = history.history['loss']
    val[str(i)] = history.history['val_loss']
# plot train and validation loss across multiple runs
pyplot.plot(train, color='blue', label='train')
pyplot.plot(val, color='orange', label='validation')
pyplot.title('model train vs validation loss')
pyplot.ylabel('loss')
pyplot.xlabel('epoch')
pyplot.show()
```



Fine-tuning a LSTM



Tuning Problem Framing

Value Scaling

- Normalize values
- Standardize values

Encoding

- One-hot Encoding
- Real-value Encoding
- Other encodings

Input Sequence length

LSTM model type

Do different runs by changing these values



Tuning Model Structure

Architecture

Memory Cells

- Grid Search on the memory cell parameter to arrive at an optimal one
- Plot a Box plot to identify the best memory cell for the lowest loss

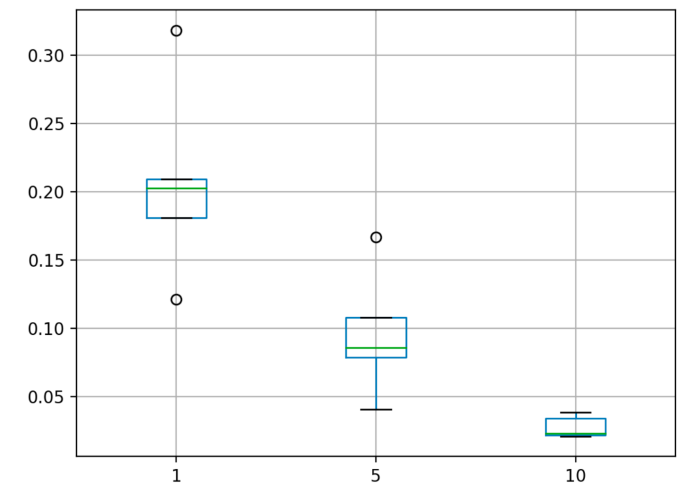
Hidden layers

Weight Initialization

- random.uniform
- random.normal

Activation functions

- sigmoid
- tanh
- relu



Tuning Learning Behaviour

Optimization algorithm

- Adam
- RMSProp
- Adagrad

Learning Rate

Batch Size

- Batch size of 1 for stochastic gradient descent.
- Batch size of n , where n is the number of samples for batch gradient descent.
 - **Grid search batch sizes in powers of 2 from 2 to 256 and beyond.**

Dropout

- dropout: dropout applied on input connections.
- recurrent dropout: dropout applied to recurrent connections.
 - **Grid search different dropout percentages.**
 - **Experiment with dropout in the input, hidden, and output layers**



Tuning Learning Behaviour

bias regularizer: regularization on the bias weights.

kernel regularizer: regularization on the input weights.

recurrent regularizer: regularization on the recurrent weights.

- L1L2(0.0, 0.0), e.g. baseline or no regularization.
- L1L2(0.01, 0.0), e.g. L1.
- L1L2(0.0, 0.01), e.g. L2.
- L1L2(0.01, 0.01), e.g. L1L2 also called elastic net.

Early Stopping

```
from keras.callbacks import EarlyStopping  
es = EarlyStopping(monitor='val_loss', patience=100)  
model.fit(..., callbacks=[es])
```



Further Reading

LSTM

<http://karpathy.github.io/2015/05/21/rnn-effectiveness/>

<http://arxiv.org/pdf/1503.04069.pdf>

<http://jmlr.org/proceedings/papers/v37/jozefowicz15.pdf>

